

## Auditing by session

The first way of using PgAudit (and the most simple to understand and try) is by session.

As with other configuration settings, you can also set the PgAudit configuration by means of the SET SQL statement. This is useful to test the configuration before applying it to the whole cluster. Try setting `pgaudit.log` directly in your interactive session and perform some actions to see what happens. As an example, suppose we want to audit any changes to the data:

```
forumdb=# SET pgaudit.log TO 'write, ddl';
SET
forumdb=# SELECT count(*) FROM forum.categories;
count
-----
      3
(1 row)

forumdb=# INSERT INTO forum.categories( description, title ) VALUES(
'Fake', 'A Malicious Category' );
INSERT 0 1

forumdb=# SELECT count(*) FROM forum.categories;
count
-----
      4
(1 row)

forumdb=# INSERT INTO forum.categories( description, title ) VALUES(
'Fake2', 'Another Malicious Category' );
INSERT 0 1
```



The `pgaudit.log` parameter can be set only by superusers, therefore if you want to try it dynamically in an interactive session, you need to connect as a database administrator. You can, of course, set this for all users at a cluster-wide level by setting the parameter in the `postgresql.conf` configuration file.

In the logs, PostgreSQL will write something like the following:

```
LOG:  AUDIT: SESSION,1,1,WRITE,INSERT,,, "INSERT INTO forum.categories(
description, title ) VALUES( 'Fake', 'A Malicious Category' );", <not
logged>
LOG:  AUDIT: SESSION,2,1,WRITE,INSERT,,, "INSERT INTO forum.categories(
description, title ) VALUES( 'Fake2', 'Another Malicious Category'
);", <not logged>
```

There are several details in such a log line, but before we examine the fields, please note that nothing has been written about the two `SELECT` statements: since we asked `PgAudit` to not audit `READ` queries, the `SELECT` statements have been discarded from auditing.

Please note that every audit line has a quite self-explanatory prefix, `AUDIT`, which makes it simple to understand whether the log line has been produced by `PgAudit` or by some other event internal to PostgreSQL.

Every line indicates the type of auditing—in the preceding example, `SESSION`—and a counter that increments to indicate the chronological order in which statements have been audited. Then there is the category of statement that `PgAudit` recognize—in the preceding, both are `WRITE` event—and then follows the complete statements that have been executed. There is room for other details, which will be discussed in further examples.

Let's move on with another example—consider the execution of a dynamically built query like the following one:

```
forumdb=# DO $$ BEGIN
EXECUTE 'TRUNCATE TABLE ' || 'forum.tags CASCADE';
END $$;
NOTICE: truncate cascades to table "forum.j_posts_tags"
DO
```

Instead of executing a `TRUNCATE TABLE tags` statement, the statement has been built by concatenating two strings. In the logs, `PgAudit` inserts a line as follows:

```
LOG:  AUDIT: SESSION,3,1,WRITE,TRUNCATE TABLE,,, TRUNCATE TABLE forum.tags
CASCADE, <not logged>
```

Again, the line reports the auditing mode (SESSION), the auditing statement number (3), the category (WRITE), and the statement (TRUNCATE TABLE), as well as the fully executed statement. This last detail is important: if you execute the same statement without auditing, PostgreSQL logs will contain a line as follows:

```
LOG:  duration: 12.616 ms  statement: DO $$ BEGIN
      EXECUTE 'TRUNCATE TABLE ' || 'forum.tags CASCADE';
    END $$;
```

Here, you can see the logs have blindly copied the source statement, including string concatenation and newlines, making it difficult to read and search for.

## Auditing by role

The auditing-by-role mechanism of PgAudit allows you to define in a very fine-grained way what events you are interested in auditing.

The idea is that you define a database role and grant permissions related to the action you want to audit to the role. Once the role and its permissions are set, you inform PgAudit to audit by that role, which means PgAudit will report in the logs any action that matches the one granted to the auditing role without any regard to the role that performed it.

The first step is therefore the creation of a role that is used only to specify which actions to audit, and therefore will not be used as an ordinary role for interactive sessions:

```
forumdb=# CREATE ROLE auditor WITH NOLOGIN;
CREATE ROLE
```

In order to specify which actions the role must audit, we simply have to GRANT those to the role. For example, assuming we want to audit all DELETE actions on every table and INSERT actions only on posts and categories, we have to grant the role the following set of permissions:

```
forumdb=# GRANT DELETE ON ALL TABLES IN SCHEMA forum TO auditor;
GRANT
forumdb=# GRANT INSERT ON forum.posts TO auditor;
GRANT
forumdb=# GRANT INSERT ON forum.categories TO auditor;
GRANT
```

Everything is now prepared for PgAudit to do its job, but it is fundamental that the auditing system knows that the auditor role has to be used, therefore we need to configure `pgaudit.role` either in the cluster configuration or in the current session. The former method is, of course, the right one to use with a production environment, while setting the configuration parameter in a single session is useful for testing purposes. Let's set the parameter in the session as a database administrator to test it in action:

```
forumdb=# SET pgaudit.role TO auditor;
SET
```

Now it is time to execute a few statements and see what PgAudit stores in the cluster logs:

```
forumdb=# INSERT INTO forum.categories( title, description ) VALUES(
'PgAudit', 'Topics related to auditing in PostgreSQL' );
INSERT 0 1

-- this will not be logged
forumdb=# INSERT INTO forum.tags( tag ) VALUES( 'pgaudit' );
INSERT 0 1

forumdb=# DELETE FROM forum.posts WHERE author NOT IN ( SELECT pk FROM
forum.users WHERE username NOT IN ( 'fluca1978', 'sscotty71' ) );
DELETE
```

As you can imagine, PgAudit will log the first and last statements of the preceding example session: in fact, only those statements are related to tables and actions the auditor role has been granted. In the PostgreSQL logs, you will find something similar to the following lines:

```
LOG:  AUDIT: OBJECT,1,1,WRITE,INSERT,TABLE,forum.categories,"INSERT INTO
forum.categories( title, description ) VALUES( 'PgAudit', 'Topics related
to auditing in PostgreSQL' );",<not logged>
LOG:  AUDIT: OBJECT,2,1,WRITE,DELETE,TABLE,forum.posts,"DELETE FROM forum.
posts WHERE author NOT IN ( SELECT pk FROM forum.users WHERE username NOT
IN ( 'fluca1978', 'sscotty71' ) );",<not logged>
```

Please note that the tuple insertion against the tags table is missing: it has not been audited and logged because the auditor role does not include a specific GRANT permission for it.

Once our auditing role has been properly configured, we can save the configuration after modifying the configuration file, `postgresql.conf`, and setting the `pgaudit.role` tunable as follows:

```
pgaudit.role = 'auditor'
```

As you can see, role-based auditing is much more flexible than session-only-based: while the latter allows you to specify only the categories of actions to audit, the former allows the fine-grained definition of exactly which statements to audit.

## Summary

PostgreSQL provides a reliable and flexible infrastructure for logging that allows a database administrator to monitor what the cluster has done in the very near past. Thanks to its flexibility, the logs can be configured to allow access by external tools for cluster analysis, such as pgBadger. Moreover, the same logging infrastructure can be exploited to perform auditing, a kind of introspection often required by local government laws.

In this chapter, you have learned how to configure the PostgreSQL logging system to match your needs, how to monitor your cluster by means of the web dashboards provided by pgBadger, and finally, how to perform auditing on your users and applications.

In the next chapter, you will learn how to back up your own cluster.

## Verify your knowledge

- What is the difference between logging and auditing?

Logging is a way to track certain activities that happen within the cluster, without any particular regard to the “target” of such an activity. On the other hand, auditing is a way to log and track specific activities that happen on specific targets. For example, logging can track “every slow query” without any regard to the table the query is run against, while auditing can track “every modification to table xyz.” See the *Implementing auditing* section for more details.

- What is pgBadger?

pgBadger is an external command that can inspect the PostgreSQL textual logs and build a dashboard with the cluster activity. See the *Extracting information from logs – pgBadger* section for more details.

- How can the database administrator decide where to send PostgreSQL logs?

PostgreSQL provides a set of logging configuration parameters, for example, `log_directory` and `log_filename`, that determine where PostgreSQL will store the logs. See the *Where to log* section for more details.

- What is the logging collector?

The logging collector is a special PostgreSQL process that collects the log that the daemon writes on standard error, and then redirects that information to the appropriate location (e.g., a file on disk). See the *Where to log* section for more details.

- What is the pgAudit extension?

PgAudit is an extension that allows the auditing of particular queries in either a session or user mode. See the *Configuring PostgreSQL to exploit PgAudit* section for more details.

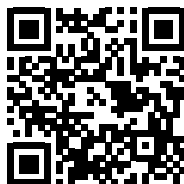
## References

- The pgBadger official documentation, available at <https://pgbadger.darold.net/documentation.html>
- PgAudit official code repository, available at <https://github.com/pgaudit/pgaudit>
- The PostgreSQL log settings' official documentation, available at <https://www.postgresql.org/docs/current/runtime-config-logging.html>
- PgAudit official website and documentation, available at <https://www.pgaudit.org/>

## Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>





# 15

## Backup and Restore

It doesn't matter how solid your hardware and software is – sooner or later, you will need to go back in time to recover accidentally deleted or damaged data. That is the purpose of a backup – to provide a safe copy that you can keep for a specific amount of time that allows you to recover from data loss. Being an enterprise-level database cluster, PostgreSQL provides a set of specific tools that allow a database administrator to take care of backups and restorations, and this chapter will show you all the main tools that you can exploit to be sure your data will survive any accidental abuse.

Backup and restore isn't a very complex topic, but it's fundamental in any production system and requires careful planning. In fact, with a backup copy, you are holding another exact copy of your database just in case something nasty happens; this extra copy will consume resources, most notably storage space. Deciding how many extra copies, how frequently you collect them, and how long they must be kept is something that requires careful attention and is beyond the scope of this chapter. In this chapter, we will look at the main ways of performing a backup, either logically or physically, and all the tools that a PostgreSQL distribution provides so that you can manage backups.

In this chapter, we will cover the following topics:

- Introducing various types of backups and restores
- Exploring logical backups
- Exploring physical backups

## Technical requirements

You need to know about the following to complete this chapter:

- How to interact with command-line tools
- How to inspect your filesystem and the PGDATA directory

The chapter examples can be run on the `chapter_15` Docker image, which you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>. For installation and usage of the Docker images available for this book, please refer to the instructions in *Chapter 1, Introduction to PostgreSQL*.

## Introducing types of backups and restores

There are mainly two types of backups that apply to PostgreSQL: the **logical backup** and the **physical backup** (also known as a **hot backup**). Depending on the type of backup you choose, the restore process will differ accordingly.

PostgreSQL ships all the integrated tools to perform the classical logical backup, which in most cases suffices. However, PostgreSQL can be easily configured to support physical backups, which are useful when the size of the cluster becomes huge, as well as when you have particular needs, as you will discover later in this chapter.

But what is the difference between these two backup methods? As you can imagine, they both achieve the very same aim: allowing you to get a usable “copy” of your data to restore it somewhere. The difference between the two backup strategies comes from the way data is extracted from the cluster.

A logical backup works similarly to a database client that asks for all the data in a database, table by table, and stores the result in a storage system. It is like an application opening a transaction and performing `SELECT` on every table, saving the obtained data on disk. Of course, it is much more complex than that, but this example gives you a simple idea of what happens under the hood.

This kind of backup is “logical” because it runs alongside other database connections and activities, as a dedicated client application, and relies on the database to provide data that is “logically” consistent. In fact, the backup is executed within a snapshot of the database to keep data consistent.

The advantages of this backup strategy are that (i) it is simple to implement since PostgreSQL provides all the software to perform a full backup, (ii) it is consistent, and (iii) it can be restored quite easily.

However, this backup method also has a few drawbacks: being performed alongside other database activities, it can slow down (or can be slowed down) together with other active concurrent transactions. Moreover, it requires the database to keep track of the ongoing backup process without trashing the snapshot as long as the backup is running. Lastly, the produced backup set is consistent at the time the backup has started; that is, if the backup requires a very long time to complete, data changes that occurred in the meantime might not be present in the backup (because it has to be consistent).

A physical backup, on the other hand, is less invasive with regard to other connections and transactions: the backup requires a file-level copy of the PGDATA content – mainly the database file (PGDATA/base) and the **Write Ahead Logs (WALs)** from the backup’s start instance to the backup’s end. The result will be an inconsistent copy of the database that needs particular care to be restored properly. Essentially, the restore will proceed as if the database has crashed and will redo all the transactions (extracted from the WALs) in order to achieve a consistent state.

This kind of backup is much more complex to set up, and while you can perform it on your own, as you will see in this chapter, several tools have emerged to help you perform this kind of backup in a more proficient and reliable way. The main advantage of this kind of backup strategy is its less invasive nature – the database is not going to notice any particular activity related to the backup except for the storage I/O bandwidth required to perform the file-level copy. Another important advantage of this backup strategy is that it allows for **point-in-time recovery (PITR)**, which allows a database administrator to recover the database to any instance since the original backup.

There is another consideration to take into account when designing for a backup strategy: logical backups are supposed to always work, regardless of the database version you are running (assuming you are running the latest version already) and, to some extent, regardless of whether the target database is PostgreSQL. On the other hand, physical backups will only work between the very same major versions of PostgreSQL instances and operating system architecture.

In the next section, you will learn how to perform both backup methods, as well as how to restore a backup. We’ll start with logical backups first.

## Exploring logical backups

PostgreSQL ships with all the required tools to perform a logical backup and restore. Many operating systems, including FreeBSD and Debian GNU/Linux, provide scripts and wrappers for the PostgreSQL backup and restore tools to ease the system administrator in scheduling backups and restores. Such scripts and wrappers will not be explained here. For more information, consider reading your operating system’s PostgreSQL package documentation.